Extension OPTIM

Groupe 6, équipe 34

January 2025

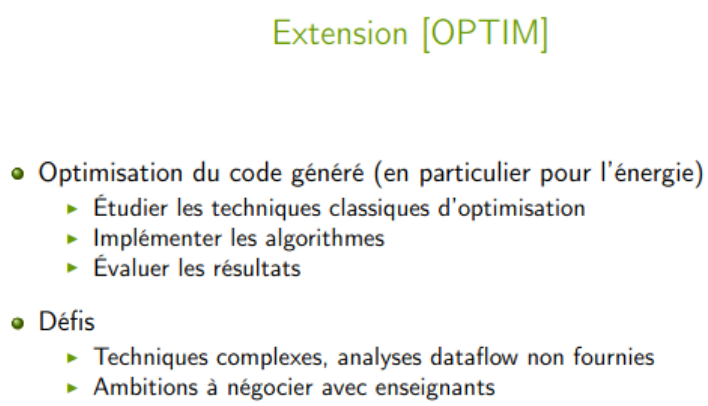

Figure 1: Slide de présentation d'OPTIM

Contents

1 Introduction **2**
1.1 Utilisation de l'extension 3
1.2 État de l'avancement 3

2 Conception **3**
2.1 Static Single Assignment form 3
2.1.1 Phi functions 4
2.1.2 Pruned SSA 5
2.2 Control Flow Graph 5
2.3 Structure des données pour l'optimisation 6
2.3.1 Qu'est ce qui est représenté 6
2.3.2 GraphSSA 6
2.3.3 AbstractBloc 6
2.3.4 InstBloc extends Bloc 6
2.3.5 IfBloc extends AbstractBloc 7

2.3.6 WhileBloc extends Bloc . . . 7
2.3.7 JoinBloc extends AbstractBloc . . . 7
2.3.8 EntryBloc extends AbstractBloc . . . 7
2.3.9 ExitBloc extends AbstractBloc . . . 7
2.3.10 lastUsedVars . . . 7
2.3.11 Class SSAVariable extends AbstractIdentifier . . . 8
2.4 Construction de la structure de données pour l'optimisation . . . 8
2.4.1 Le graphe . . . 8
2.4.2 Itérer sur ListInst . . . 8
2.4.3 Cas if . . . 8
2.4.4 Cas While . . . 8
2.4.5 Cas Assign . . . 9
2.4.6 Placement des phi functions . . . 9
2.5 Algorithme . . . 9
2.5.1 Construction du graphe . . . 9
2.6 Remplacement des Identifier par des SSAVariables . . . 10

3 Listes d'optimisations faisable 10
3.1 Propagation de constante . . . 10
3.2 Simplification de calcul . . . 10
3.2.1 Optimiser la gestion des registres dans les opérations . . . 10
3.2.2 Élimination de calcul constant . . . 10
3.2.3 L'opération FMA . . . 11
3.3 Mémoïsation . . . 11
3.4 Variables déjà chargée . . . 11

4 Analyse bibliographique 11

5 Méthode de validation 12
5.1 Validation de la construction du graphe . . . 12
5.2 Validation de l'insertion des SSAVariables dans les instructions . 12
5.3 Résultats de la validation . . . 12

6 Conclusion 12

7 Bibliographie 13

1 Introduction

Le but de cette extension est d'optimiser le code généré. Notre but est d'ajouter une étape dans notre compilateur (entre l'étape B et C), durant laquelle la représentation du programme change pour devenir un control flow graph, et avoir notre programme dans la forme SSA (static single assignment). Ces changements n'optimisent pas le code généré, mais cette représentation du code simplifie des optimisations.
Cette nouvelle représentation du code est détaillée dans les parties suivantes,

ainsi que les optimisations.
Notez que ce document a servi de base a tout le groupe pour ce mettre d'accord sur la meilleure manière d'implémenter l'extension, et a donc évolué au cours du projet.

1.1 Utilisation de l'extension

L'extension n'est pas utilisée par défaut. Il faut spécifier, avec l'option "-o" au compilateur qu'on souhaite qu'il optimise le code généré.
Ce choix à été fait pour des raisons pratiques :

- **Robustesse du compilateur** : le compilateur sans optimisation est toujours disponible, et permet la création d'un code de référence lors de l'implémentation de l'extension.
- **Écologie** : l'optimisation de code consomme beaucoup de ressources, par rapport au compilateur d'origine. De plus, les optimisations implémentés pour l'instant ne justifie pas cette consommation de ressources supplémentaire. Le choix est donc laissé à l'utilisateur d'utiliser cette option ou non.

1.2 État de l'avancement

Une majeure partie de l'extension a été implémentée après le dernier rendu, et n'a donc pas bénéficier de beaucoup de temps de programmation.
En l'état actuel, l'extension parait décevante puisque le code que nous générons est largement moins efficace que celui de référence du compilateur des professeurs (envions 30% plus de cycles nécessaires à l'exécution).
Cependant, le travail réalisé n'est pas anodin. La plus grande partie du travail sur cet extension à été un travail de documentation, et de représentation interne du code pour le compilateur. La représentation interne du cod est ce qui est détaillé dans la suite de ce document, dans les 3 parties suivantes.
La seule optimisation faite au moment du rendu est la propagation de constante, un optimisation simple qui consiste à remplacer les variables par leurs valeurs, si celle-ci pour être connue en avance.

2 Conception

2.1 Static Single Assignment form

SSA est une solution au problème suivant : dans notre programme, nous ne pouvons pas savoir ce qu'il y a dans une variable. Le programme peut modifier une valeur, et même en changer le type. Alors les simplifications avec les variables sont difficiles à mettre en place.
L'idée est que chaque variable ne se voit assigner une valeur qu'une seule fois, et donc que sa valeur ne change jamais (dans la représentation qu'on fait du

```
I ← 1                      I1 ← 1
J ← 1                      J1 ← 1
K ← 1                      K1 ← 1
L ← 1                      L1 ← 1
repeat                     repeat
                              I2 ← φ(I3,I1)
                              J2 ← φ(J4,J1)
                              K2 ← φ(K5,K1)
                              L2 ← φ(L9,L1)
   if (P)                     if (P)
      then do                    then do
         J ← I                      J3 ← I2
         if (Q)                     if (Q)
            then L ← 2                 then L3 ← 2
            else L ← 3                 else L4 ← 3
                                    L5 ← φ(L3,L4)
         K ← K + 1                  K3 ← K2 + 1
      end                        end
      else K ← K + 2             else K4 ← K2 + 2
                              J4 ← φ(J3,J2)
                              K5 ← φ(K3,K4)
                              L6 ← φ(L2,L5)
   print(I,J,K,L)             print(I2,J4,K5,L6)
   repeat                     repeat
                                 L7 ← φ(L9,L6)
      if (R)                     if (R)
         then L ← L + 4             then L8 ← L7 + 4
                                 L9 ← φ(L8,L7)
   until (S)                  until (S)
   I ← I + 6                  I3 ← I2 + 6
until (T)                  until (T)
```

Fig. 6 Simple program and its SSA form.

Figure 2: Illustration tiré de : [6]Exemple de programme sous forme SSA

programme).
Pour un programme qui aurait alors une variable A qui changerait de valeur en cours d'exécution, on considérera dans notre représentation les variables $A_1,\ A_2, ...,$ une pour chaque assignation de valeur pour la variable A.
Il est à noter que toutes les variables se voient assigner une valeur par défaut au début du programme.

2.1.1 Phi functions

Lorsque l'on se trouve dans un bloc **join** (comprendre ici que plusieurs nœuds sont ses prédécesseurs directs, voir figure 3), on ne peut pas être certain des valeurs de nos variables. La solution est donc de considérer qu'il ne s'agit d'aucune des deux, et donc de faire une assignation avec une ϕ **function** (**Pour l'instant**, cette fonction n'a aucun intérêt pratique, elle est juste là pour signifier

ce qui peut se trouver dans la nouvelle variable)

2.1.2 Pruned SSA

Cette partie n'a pas été utilisée. L'idée est d'optimiser le placement des phi functions, pour qu'elles n'apparaissent plus pour les variables qui ne sont plus utilisées plus tard dans le programmes. On parle de variables **"vivantes"** ou **"mortes"**.

2.2 Control Flow Graph

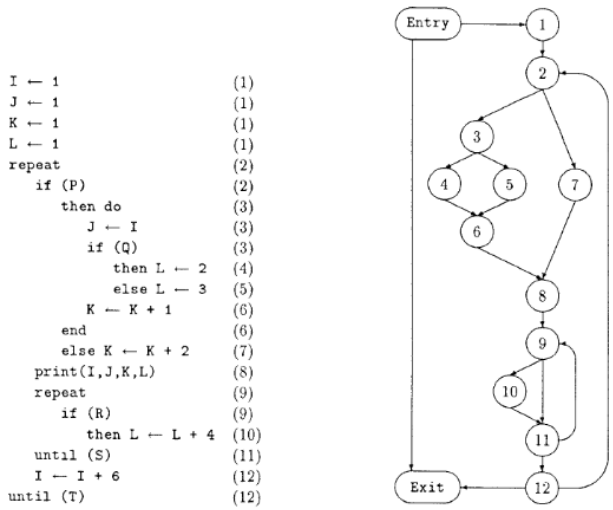

Figure 3: Illustration tiré de : [6]Exemple de graphe et son programme associé

L'objectif du graphe de contrôle de flow est de rendre plus évidents les différents chemins que le programme peut parcourir.
Les différentes instructions qui terminent un bloc sont :

- **if** donne lieu à un nœud branche (deux nœuds accessibles après).
- **else** s'il existe, met fin au bloc **then**.
- **end if** met fin au bloc **else** s'il existe, sinon met fin au bloc **then**.
- **while** commence le bloc. Il est terminé par la fin de boucle.
- **assign** met fin à un bloc aussi. C'est un choix d'implémentation, pour que chaque bloc puisse avoir un seul environement (i.e. une seule représentation de chaque variable).

Dans un graphe, on désignera par bloc parent un bloc du graphe qui mène au bloc voulu. De même pour fils (ou "son" dans le code), il s'agit d'un bloc qu'on atteind depuis le bloc voulu.

2.3 Structure des données pour l'optimisation

Cette section est une première idée de ce que pourrait être l'implémentation, et a plus pour but d'aider à la compréhension générale de ce que doit être l'extension plutôt que de base pour le code.

2.3.1 Qu'est ce qui est représenté

Un graphe représente le **main** ou une **méthode**.
Plus précisément, il s'agit de représenter toutes les différentes exécutions possibles de ces lignes de code. Il y aura plusieurs graphes à construire, mais ils devraient tous suivre le même schéma et les optimisations devront pouvoir s'appliquer sur chacun de ces graphes (i.e. main et méthodes).

2.3.2 GraphSSA

Cette classe représente le graphe de contrôle de flow d'une méthode ou du main. Son constructeur prend en argument ListDeclVar (pour le EntryBloc uniquement) et la ListInst

- Bloc entry (le bloc d'entré, initialise les variables, ou les arguments de la methode)
- Bloc exit (bloc final)
- void codeGen(DecacCompiler compiler) génere le code, ne doit pas beaucoup changer de celle qu'on a déjà sur le main ou les méthodes, elle doit juste s'adapter pour aller chercher le code dans le bon ordre dans les blocs

2.3.3 AbstractBloc

Cette classe représente un bloc de code

- HashTable<Identifier (la variable), SSAVariable> lastUsedVars (permet de faire en sorte que le
- AbstractBloc parent
- AbstractBloc son
- ListInst listInst

2.3.4 InstBloc extends Bloc

Ce type de bloc est le type de base. Il possède un parent, un fils, et sa ListInst.

2.3.5 IfBloc extends AbstractBloc

L'instruction if nécessite son propre type. C'est avec le WhileBloc les deux types de bloc qui possèdent 2 fils. Il faut noter que le elseSon est vide s'il n'a pas de else.

- AbstractBloc elseSon

Les deux bloc fils (ou le IfBloc lui même s'il n'y a pas de bloc else) sont parents d'un JoinBloc.

2.3.6 WhileBloc extends Bloc

De même que pour IfBloc, l'instruction suivante est contenue dans l'objet While de l'arbre de la partie B, il est donc certainement plus simple de gérer ici la création de bloc suivant et de génération de code.
Un whileBloc n'a pas de ListInst, il ne sert qu'a redirigé vers un InstBloc, dont il est le père et le fils (boucle), ou la suite du programme.

2.3.7 JoinBloc extends AbstractBloc

Le but de ce bloc est de faire en sorte que les différentes branches des "if" du graphe puissent converger. Il doit avoir 2 parents, éventuellement des JoinBloc, et un seul fils.
Il ne doit pas comporter de ListInst (on la laisse vide).

2.3.8 EntryBloc extends AbstractBloc

Ce bloc est le premier du graphe. Il contient uniquement un bloc fils, ainsi qu'une ListDeclVar. C'est à partir de celle-ci que l'environnement (HashTable qui associe a un variable, une SSAVariable, voir section EnvSSA)

2.3.9 ExitBloc extends AbstractBloc

Ce bloc est le dernier du graphe, il le termine. Il pourra avoir différentes utilités en fonction de si le graphe représente une méthode ou le main. Il n'a qu'un seul parent, et aucune instruction.

2.3.10 lastUsedVars

Il s'agit d'une HashTable<Symbol, SSAVariable>. Elle retient quelles sont les dernières SSAVariable associées aux différentes variables, donc celles qui s'appliquent dans le bloc actuel.
Chaque bloc devra avoir le sien, hérité de son parent, modifié en cas d'instruction assign.
Les WhileBloc et les JoinBloc sont les seuls à avoir deux parents, mais pas de ListInst, leur seul fonction est donc de s'occuper des **phi functions**.
lastUsedVars n'est utilisé que pour le remplacement dans Variables par les

SSAVariables, où les SSAVariables par les nouvelles SSAVariables, au moment de passe de corrections, en fonction du calcul des phi functions ou d'optimisations.

2.3.11 Class SSAVariable extends AbstractIdentifier

Cette classe représentes les différentes variables dans notre représentations SSA du programme.
Elle étend Identifier pour avoir toutes les propriétés

- Variable variable (pointe vers la variable qu'elle représente)
- Liste des (Variable) variables modifiés

2.4 Construction de la structure de données pour l'optimisation

La construction du graphe est à faire pour chacune des méthodes, ainsi que pour le main.

2.4.1 Le graphe

Le graphe en lui même ne contient que les deux blocs **entry** et **exit**.
Il pourra y avoir d'autres données pour les optimisations.

2.4.2 Itérer sur ListInst

Une méthode sur les InstList parcourt les instructions, en les ajoutant les une à la suite des autres dans une InstList, qui sera affectée au Bloc courant.
Cette méthode prend en paramètre un Bloc précédant, et un Bloc suivant.
Chaque bloc connaît son bloc précédant et son bloc suivant. Dans le cas du premier bloc, il s'agit des bloc **entry** et **exit**.

2.4.3 Cas if

Si l'opérations sur laquelle on arrive est un if, on crée 1 bloc, celui de **endIfBloc**.
Ce bloc est le bloc suivant de **ThenBloc** et **ElseBloc** (si ce 2e existe, sinon endIfBloc remplace ElseBloc).
La suite est généré automatiquement en appelant la méthode sur les InstList qui créer le graphe, cette fois-ci avec le bloc courant comme précédant, et le endIfBloc généré comme suivant.
Il faut faire la même chose pour le ElseBloc, **ATTENTION au cas où il n'y a pas d'instruction dans la elseBranch, à voir pour que genCode fonctionne le mieux possible.**

2.4.4 Cas While

Ce cas se comporte comme le cas if, à l'exception près que le bloc est son propre prédécesseur.

2.4.5 Cas Assign

Dans le cas où cette opération est rencontré, il faudra faire appelle à la fonction newSSAVariable de la variable associé en question, et changer dans la liste des SSAVariables finales du bloc la SSAVariable en question.

2.4.6 Placement des phi functions

Lorsque l'on débute un bloc, tous les prédécesseurs sont créés. Il est donc possible de faire une comparaison des listes de SSAVariable finale des blocs précécesseurs (s'il y en a plusieurs).
Dans le cas ou on ne détecte aucune différence, on garde ces listes de SSAVariable pour ce bloc, qui pourront être amenée à changer au cours des prochaines instructions lues.
Dans le cas contraire, on utilise la fonction newSSAVariable pour fabriquer une nouvelle SSAVariable, qui sera utilisée ensuite.

2.5 Algorithme

2.5.1 Construction du graphe

Construire notre graphe et remplacer les variables par des SSAVariables dans les instructions se fait en une seule passe. La méthode principale est **genGraph** de la class ListInst. C'est sur la liste d'instruction du main, ou d'une méthode, que cette méthode est appelée.
Elle itère sur les instructions, jusqu'à un des cas suivant :

1. **Fin de la liste d'instruction** : la méthode génère simplement un InstBloc, qui relie le bloc entry au bloc exit.

2. **IfThenElse** : on créer un IfBloc ainsi qu'un JoinBloc, qui serviront de bloc d'entré et de bloc de sortie aux appelle de la méthode sur les listes d'instructions de la partie Then et de la partie Else. Une fois que cela est fait, la méthode change sa variable entry pour le JoinBloc généré, et fait la suite comme avant l'instruction précédante.

3. **While** : Dans le cas d'un While, on créer un WhileBloc, pour pouvoir appeller la méthode sur la liste d'instruction du body du while, avec le WhileBloc comme entré et comme sorti. Une fois que cela est fait, la méthode change sa variable entry pour le WhileBloc généré, et fait la suite comme avant l'instruction précédante.

Il est à noté que le fils d'un bloc n'est mis en place que lorque le fils met en place son parent.

2.6 Remplacement des Identifier par des SSAVariables

Le programme passe sur chacune des instructions, et remplace chacun des Identifier par la dernière SSAVariable associé. C'est la méthode **transformSSAInst** qui s'occupe de cette tâche. Le remplacement de l'Identifier se fait en fonction de ce qu'on a vu plus tôt dans le programme : cela est représenté par **lastUsedVars**, une HashTable qui associe une SSAVariable (celle créée par le dernier Assign croisé) à un Identifier (clé de la HashTable).

3 Listes d'optimisations faisable

3.1 Propagation de constante

Cette optimisation est à faire en premier pour permettre aux optimisation de calcul de prendre en compte ce qu'on fait ici
Si on détecte qu'une variable est finalement constante (pas de réaffectation de valeur), on peut remplacer cette variable par sa valeur, si elle est connue.
Cette optimisation est aussi à faire pour les booléens

3.2 Simplification de calcul

L'exemple le plus classique est la division/multiplication par une puissance de 2, un shift right/left est plus simple a faire pour la machine.
Il existe plusieurs simplifications de ce genre, il faudra voir ce qu'on aura le temps de faire.

3.2.1 Optimiser la gestion des registres dans les opérations

La gestion actuelle des registres est la plus naïve qui soient, n'utilisant que les resgistres **R1** et **R0** pour les calculs, le reste ne sert qu'a stocker des données.
L'idée est qu'il faudrait faire les calculs directement sur les registres où les données sont stockées, pour s'épargner les opérations **LOAD** et **STORE** pour chacun des deux opérandes.
Cette optimisation sera sûrement à faire dans la fonction codeGenBinaryExp de la class AbstractBinaryExpr.
Cette optimisation est aussi à faire pour les booléens

3.2.2 Élimination de calcul constant

On va chercher a calculer en avance les calculs qui n'utilisent que des constantes, pour que le programme n'ai pas à le faire.
On peut aussi faire en sorte que les calculs triviaux soient éliminés :

- multiplication par 0 ou 1
- addition ou soustraction avec 0

3.2.3 L'opération FMA

L'opération assembleur **FMA** permet de faire une multiplication et une addition, tout en gagnant du temps si les opérations étaient faites séparément.
Cette optimisation est aussi à faire pour les booléens
Cette optimisation est à coupler avec la suppression de branches "mortes" (inaccessibles).

3.3 Mémoïsation

Des calculs peuvent être amenés à être faits plusieurs fois au cours de l'exécution d'un programme. Il peut s'agir d'un calcul ou d'un appel de fonction avec les mêmes paramètres (faire attention à ce cas, une fonction peut modifier des objets/variables, son intérêt n'est pas toujours de return un valeur). La section simplification de calcul est nécessaire pour cette partie, pour avoir des expressions reconnaissable plus facilement (a+b est le même calcul que b+a, comment le voir? On ne peut pas normaliser toutes les opérations (ordre alphabétique par exemple) puisque la sémantique de déca impose d'évaluer de gauche à droite les expressions).
L'idée est de repérer les opérations qui sont répétées, pour que le code assembleur affecte une nouvelle variable dont on récupérera la valeur, plutôt que de refaire le calcul.
Une HashTable avec l'opération ou la méthode comme clé, transformée en String, semble être une implémentation commune.

3.4 Variables déjà chargée

Si une variable est utilisée plusieurs fois, il peut être intéressant d'avoir un registre bloqué spécifiquement pour elle, pour qu'elle n'ai pas à être chargée à chaque appelle.

4 Analyse bibliographique

Le travail sur l'extension à commencé par de simple recherche, sur ce qui existait comme solution d'optimisation de code généré. La première ressource, qui à inspiré la partie Liste d'optimisation faisable, est la page du site internet [1]Zeste de savoir.
Sur les conseils de notre professeur, nous nous sommes intéressés à la représentation SSA d'un programme (static single assignment). La page [3]Wikipedia associé nous à servie de base pour la compréhension de ce nouveau principe. Parmis les sources de cette page, nous avons trouver un document; [6]"Efficiently computing static single assignment form and the control dependence graph" par Cytron, Ron; Ferrante, Jeanne; Rosen, Barry K. Wegman, Mark N. Zadeck, F. Kenneth; du quel nous avons repris des illustrations pour ce document, qui a beaucoup inspiré tous nos choix de conceptions.
Un autre document est venue nous aidé a avoir une meilleure compréhension

des différents algorithmes et solution d'implémentation de la représentation SSA d'un programme,
Une piste que nous avons trouvée interessante de garder, bien que nous n'ayont pas eu l'occasion d'en faire quelquechose, est un document sur[2] les optimisations énergétique du code généré. Elle aurait pu être intéressante à combiner avec une extension de portage du code généré pour une GAMEBOY.

5 Méthode de validation

5.1 Validation de la construction du graphe

Le premier problème était d'être capable de rajouter l'étape de création de graphe, sans casser tout le reste. La validation de la construction du graphe s'est fait à l'aide de la génération de code, puisque sans optimisation, le même programme devait être généré que sans l'option "-o".

5.2 Validation de l'insertion des SSAVariables dans les instructions

Nous avons remplacer les Identifier de variables dans les instructions, dans le graphe. Nous avons fait le choix de ne pas encombrer les SSAVariables d'un nombre, pour signifier le nombre d'Assign sur un même variable, nous nous sommes contentés de comparer les pointeurs pour savoir si les remplacements se faisaient correctement, ainsi que des commentaires ajouté dans le code généré pour savoir ce qu'il se passe sur de petits exemples.

5.3 Résultats de la validation

La création du graphe ainsi que le remplacement des SSAVariables est pratiquement correct, et la propagation de constante nous l'a fait remarquer. En effet, il y a un cas pour lequel la gestion des SSAVariables comporte des erreurs, c'est celui des boucles impbriquées. Nous n'avons pas eu le temps de trouver pourquoi, mais en dehors de ce cas particulier, l'optimisation permet déjà de gagner quelques cycles à l'exécution.

6 Conclusion

La majeure partie du travail fait pour cette extension ne permet pas en soi de rendre le code généré plus efficace, mais la nouvelle représentation du code que nous avons permet d'obtenir de grands gains de performance en peu de temps, les optimisations étant beaucoup plus rapide à implémenter.
Un problème que nous avons aussi rencontré est la gestion naïves de nos registres. En effet, le majeure perte de performance du code que nous générons viens des LOAD et STORE, bien trop nombreux. Nous ne nous sommes pas avanturer à

refaire cette partie, n'étant pas en lien avec ce que nous souhaitions faire pour notre extension.

7 Bibliographie

[1] Idées d'optimisations :
Auteur anonyme
https://zestedesavoir.com/tutoriels/427/les-optimisations-des-compilateurs/
[2] Optimisation énergétique :
Olivier Zendra. Compilation et faible consommation. 2006. inria-00113532
https://inria.hal.science/inria-00113532/PDF/compil_faible_conso_tsi_zendra.pdf
[3] Wikipedia de SSA :
https://en.wikipedia.org/wiki/Static_single-assignment_form
[4] Algorithme pour le SSA :
Lots of Authors, SSA-based Compiler Design. Springer, 2018. https://pfalcon.github.io/ssabook/latest/book-full.pdf
[5] Cours sur le SSA (comprendre les bases) :
https://www.cs.cmu.edu/~fp/courses/15411-f08/lectures/09-ssa.pdf
[6] Algorithmes pour le SSA :
Cytron, Ron; Ferrante, Jeanne; Rosen, Barry K.; Wegman, Mark N.; Zadeck, F. Kenneth (1 October 1991). "Efficiently computing static single assignment form and the control dependence graph". ACM Transactions on Programming Languages and Systems. 13 (4): 451–490. doi:10.1145/115372.115320. S2CID 13243943.

https://dl.acm.org/doi/10.1145/115372.115320